

Cours de Programmation en C

Travaux Pratiques

— TP3 —

1 Un peu d'Histoire...¹

Le cylindre de Jefferson est un système de chiffrement polyalphabétique inventé par Thomas Jefferson (futur président des États-Unis) vers 1793 alors qu'il était secrétaire d'État de George Washington.

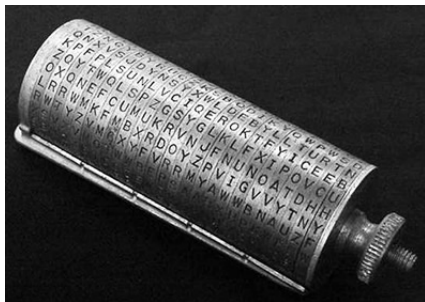


Figure 1: Cylindre de Jefferson²

Le principe de fonctionnement du cylindre de Jefferson (*Jefferson's wheel cipher* en anglais) est relativement simple et ingénieux. Il se compose d'un axe autour duquel tourne un certain nombre de roues (Figure 1). Sur la tranche de chacune d'elles, vingt-six lettres, représentant l'ensemble des lettres de l'alphabet latin, sont gravées de façon aléatoire. En tournant les roues les unes par rapport aux autres, il est alors possible d'aligner une suite de caractères de manière à composer un message. Par exemple, un expéditeur désirant écrire le message suivant "Thomas Jefferson wheel cipher", composera sur une même ligne du cylindre, l'ensemble des caractères du message. Pour coder ce dernier, l'expéditeur envoie la succession de caractères d'une ligne située au dessus ou en dessous de la ligne du message. Le destinataire recevra, par exemple, le message suivant "MZNCSKYONSLKTRFAJQQBRTXYUKA" qu'il composera sur

son cylindre identique à celui de son correspondant. En recherchant une ligne pour laquelle la succession de caractères est intelligible, le destinataire découvrira le message envoyé.

2 But du TP

Le but du TP est d'écrire un programme afin de recréer le principe de fonctionnement du cylindre de Jefferson. Ce dernier sera assimilé à un tableau 2D de 26 lignes et de N colonnes correspondant au nombre de roues du cylindre (cf figure 2). Dans la suite du TP, le nombre de colonnes N du tableau sera constant et défini avec un `#define`. On prendra $N = 19$.

j	t	s	e	c	b
f	a	h	s	q	g
:	:	:	:	:	:
:	:	:	:	:	:
y	c	b	t	z	c

Figure 2: Tableau représentation le cylindre de Jefferson

Pour chaque nouvelle question, vous devrez fournir un nouveau fichier (par exemple `Jefferson-Q1.c` pour la 1ère question). Reprenez donc le code correspondant à la question précédente (dans un nouveau fichier) pour démarrer une nouvelle question.

3 Programmation du cylindre de Jefferson

Question 1 – construction du cylindre – 6pts

¹Source wikipedia : http://en.wikipedia.org/wiki/Jefferson_disk

²Source : National Cryptographic Museum Foundation

Dans le répertoire `/home/sasl/shared/ei-ii/C/TP3`, nous fournissons un fichier `jefferson.txt` contenant les caractères d'un cylindre de Jefferson (pour 19 roues). Le fichier contient les 26 caractères de la 1^{ère} roue, suivi des 26 de la 2^{ème} roue, puis de la 3^{ème}, etc.

Écrivez deux fonctions `LitCylindre` et `AfficheCylindre` telles que :

- `LitCylindre` permet de lire les caractères contenus dans le fichier et les stocke dans le tableau. Cette fonction prendra comme paramètres :

- le nom du fichier à lire;

- le tableau à remplir.

La fonction renverra un entier indiquant si la lecture s'est bien déroulée.

- `AfficheCylindre` permet d'afficher le cylindre construit (affichage ligne par ligne).

Écrivez une fonction principale `main` qui permet de tester ces fonctions en lisant ouvrant le fichier `"jefferson.txt"` et affichant le cylindre lu.

Solution:

```

#include <stdio.h>
#include <stdlib.h>

#define N 19

/* Lit le cylindre dans un fichier (ouvert)
la lecture se fait colonne par colonne (roue par roue)
Paramètres :
- fichier: (FILE*) pointeur vers le fichier ouvert
- cylindre: (char [26][N]) cylindre de jefferson à lire*/
void litCylindre( FILE* fichier, char cylindre[26][N])
{
    int i,j;

    /* on lit le fichier roue par roue ! */
    for( j=0; j<N; j++)
        for( i=0; i<26; i++)
            fscanf( fichier, "%c", &(cylindre[i][j]) );
}

/* Affiche le cylindre, ligne par ligne
Paramètres :
- cylindre: (char [26][N]) cylindre de jefferson à afficher*/
void afficheCylindre( char cylindre[26][N])
{
    int i,j;

    /* affichage du numéro de roue */
    for( j=0; j<N; j++)
        printf( "%3d", j );
    printf("\n");

    /* on affiche la cylindre, ligne par ligne */
    for( i=0; i<26; i++)
    {
        for( j=0; j<N; j++)
            printf( "␣%c", cylindre[i][j] );
        printf("\n");
    }
}

/* programme principal */
int main()
{
    char cylindre[26][N];
    FILE* fichier;

    /* ouverture du fichier */
    fichier = fopen("jefferson.txt", "r");
    if ( !fichier )
    {
        printf( "impossible_d'ouvrir_le_fichier\n");
        exit(1);
    }

    /* lecture de la cylindre */
    litCylindre( fichier, cylindre);

    /* affichage */
    afficheCylindre( cylindre);

    /* fermeture du fichier */
    fclose( fichier);

    return EXIT_SUCCESS;
}

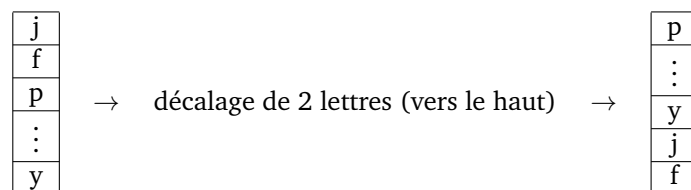
```

Question 2 – Manipulation du cylindre – 6pts

Écrivez deux fonctions `chercheLettreRoue` et `tourneRoue` telles que :

- `chercheLettreRoue` permet de déterminer dans une roue du cylindre la position (entre 0 et 25) d'une lettre cherchée (chaque lettre de l'alphabet apparait une fois par roue). Cette fonction prendra comme paramètres :
 - le tableau représentant le cylindre ;
 - le numéro de la roue ;
 - la lettre recherchée.

- `tourneRoue` permet de tourner une roue (vers le haut) d'un nombre précis de lettres. Avant de tourner la roue, et donc de décaler les lettres du bas vers le haut, le plus facile est d'utiliser un tableau intermédiaire pour stocker les lettres afin de ne pas perdre leur ordre d'origine.



Complétez/modifiez la fonction principale `main` en choisissant une roue et une lettre à rechercher et placez-la sur la première ligne du cylindre en utilisant la dernière fonction.

Solution:

```

/* tourne une roue du cylindre
Paramètres:
- cylindre: (char [26][N]) cylindre de jefferson
- nRoue : (int) numéro de la roue (de 0 à N-1)
- rotation : (int) de combien on tourne la roue (positif !) */
void tourneRoue( char cylindre[26][N], int nRoue, int rotation)
{
    char roue[26];
    int i;

    /* recopie de la roue */
    for( i=0; i<26; i++)
        roue[i] = cylindre[i][nRoue];

    /* recopie de la roue sur le cylindre AVEC la rotation*/
    for( i=0; i<26; i++)
        cylindre[i][nRoue] = roue[ (i+rotation)%26 ];
}

/* cherche la position d'une lettre dans une roue donnée
Paramètres:
- cylindre: (char [26][N]) cylindre de jefferson
- nRoue : (int) numéro de la roue (de 0 à N-1) où se fait la recherche
- lettre : (char) lettre à chercher
Retour :
- position de la lettre sur la roue*/
int chercheLettreRoue( char cylindre[26][N], int nRoue, int lettre)
{
    int pos = 0;
    /* on boucle jusqu'à trouver cette lettre
    (le cylindre est fait de telle sorte que chaque lettre existe sur la roue!)* */
    while (cylindre[pos][nRoue] != lettre)
        pos++;

    return pos;
}

/* programme principal */
int main()
{
    char cylindre[26][N];
    FILE* fichier;
    int pos;

    /* ouverture du fichier */
    fichier = fopen("jefferson.txt", "r");
    if ( !fichier )
    {
        printf( "impossible_d'ouvrir_le_fichier\n");
        exit(1);
    }

    /* lecture de la cylindre */
    litCylindre( fichier, cylindre);

    /* affichage */
    afficheCylindre( cylindre);

    /* déplace une roue pour mettre le 'a' en lère position */
    pos = chercheLettreRoue( cylindre, 0, 'a');
    printf("Position_de_a_sur_la_lere_roue_=%d\n", pos);
    tourneRoue( cylindre, 0, pos);
    afficheCylindre( cylindre);

    /* fermeture du fichier */
    fclose( fichier);

    return EXIT_SUCCESS;
}

```

Question 3 – Application au chiffrement – 4pts

Constatez que le programme crée peut permettre de coder ET de décoder un message donné en paramètre d'entrée. Pour cela, dans un fichier jeffersonQ3.c (qui reprend jeffersonQ2.c), complétez la fonction main pour demander à l'utilisateur d'entrer N caractères, soit 19 caractères, (en minuscule), et effectuer les rotations nécessaires des roues afin de placer les caractères à coder sur la première ligne du cylindre.

Lorsque cette opération sera faite, vous pourrez alors décoder le message suivant : "mkcrvylhcoiahczykdk".

Solution:

On doit décrypter "cylindredejefferson" (les espaces ne sont pas compris dans le cylindre).

```

/* programme principal */
int main()
{
    char cylindre[26][N];
    FILE* fichier;
    char code[N];
    int i, pos;

    /* ouverture du fichier */
    fichier = fopen("jefferson.txt", "r");
    if ( !fichier )
    {
        printf( "impossible_d'ouvrir_le_fichier\n");
        exit(1);
    }

    /* lecture de la cylindre */
    litCylindre( fichier, cylindre);

    /* lecture de N caractères à coder */
    printf( "Veuillez_saisir_%d_caractères_", N);
    for( i=0; i<N; i++)
        scanf( "%c", &(code[i]) );
    printf("\n");

    /* on tourne les roues comme il le faut pour former le mot sur la lère ligne */
    for( i=0; i<N; i++)
    {
        /* on trouve la position de chaque lettre à code */
        pos = chercheLettreRoue( cylindre, i, code[i] );
        /* puis on décale la roue en conséquence */
        tourneRoue( cylindre, i, pos);
    }

    /* affichage*/
    afficheCylindre( cylindre);

    /* fermeture du fichier */
    fclose( fichier);

    return EXIT_SUCCESS;
}

```

Question 4 – Ordre des roues – 2pts

En plus de cela, il est intéressant (et important) de pouvoir déterminer l'ordre des roues du cylindre car cela constitue la clé du chiffrement.

Proposez une solution où l'utilisateur doit saisir l'ordre des N roues du cylindre, puis un code à chiffrer/déchiffrer.

Question 5 – Bonus – 2pts

Ré-écrivez la fonction `tourneRoue` **sans** utiliser de tableau intermédiaire.